Abaqus CAE AI Copilot

Procedural Workflow Guide

This document is procedural guidance retrieved by FEA Planner (for shaping its plan and dispatch decisions) and Consultant (for engineering/scripting methodology advice). It is NOT a fixed script -- each section below describes the DECISION LOGIC for a category of task: which subagent to involve, in what order, what to check before dispatching, and how to recover when something is missing or fails. Real tasks often combine guidance from more than one section below (e.g. a task may need both WORKFLOW 2's named-region rule and WORKFLOW 5's job-submission sequence) -- read across sections and synthesize what genuinely applies to the specific task at hand rather than following one section rigidly.

Every workflow below reflects what this system's agents can ACTUALLY do -- their real tools, their real routing options, and their real limits. Do not extrapolate a workflow beyond what's described here into a capability the system doesn't have.

THE GOLDEN RULE: Named Regions for Any Targeted Operation

Context: this rule applies whenever a task targets a SPECIFIC sub-region of the model -- a face, surface, edge, node set, or element set -- rather than an entire part or the whole model. This is the single most important procedural rule in this document. Apply it BEFORE dispatching Coding Agent for any such task.

What counts as a "targeted operation"

Any operation that applies to a specific sub-region rather than a whole part, including but not limited to:

- Applying a load (pressure, concentrated force, moment, distributed load, etc.) to a face, surface, or node set
- Applying a boundary condition (encastre, displacement, symmetry, velocity, etc.) to a face, edge, or node set
- Defining a connection, tie constraint, coupling constraint, or contact interaction between specific surfaces
- Assigning a material or section to a specific region (rather than an entire part)
- Applying mesh seeds or mesh controls to a specific sub-region (rather than the whole part with default settings)
- Extracting or reporting a result (stress, displacement, reaction force) for a specific named node/element set, when that specific set is not already a standard, whole-part quantity

The decision procedure

Step 1 -- Identify exactly what region the user means

Read the user's description carefully. Users describe regions informally ("the top surface", "where the bracket bolts on", "the left edge") -- your job is to map this informal description to whether a matching NAMED set or surface already exists in the model, not to guess a geometric selection yourself.

Step 2 -- Check whether a matching named set/surface already exists

Use read_parsed_file_info on the relevant .inp file's derived summary yourself first -- this is a direct, cheap check you (FEA Planner) can do without dispatching anyone. The inp_summary lists defined sets, surfaces, and regions by name. Look for a set/surface name that plausibly corresponds to what the user described.

If the mapping is genuinely ambiguous or requires engineering interpretation to confirm (e.g., multiple candidate sets could plausibly be "the top surface", or you need to visually confirm which named region corresponds to the physical location described) -- dispatch Subject Matter Expert to interpret the model (it has the same read_parsed_file_info access plus the two screenshot tools, and can visually confirm which named region matches the described location) and report back which specific named set/surface, if any, matches.

Step 3A -- A clear matching named set/surface exists: proceed

Dispatch Coding Agent with dispatch_instructions that state the EXACT, VERBATIM set/surface name as it appears in the .inp file -- copy it exactly, character for character, from what read_parsed_file_info or SME returned. Never paraphrase, re-type from memory, or guess at capitalization/spacing. A subtle mismatch

(case, underscore vs. space, a trailing number) will cause Coding Agent's script to fail at runtime with a KeyError or equivalent -- and that failure is much harder for Coding Agent to self-diagnose than it is for you to get the name exactly right at dispatch time.

```
EXAMPLE dispatch_instructions excerpt:
"Apply a concentrated force of 500 N in the -Y direction to the node set named
'RP-1' (exact name, verbatim from the .inp file's defined sets) in Step-1. Do
not apply this to any other set or attempt to geometrically re-derive this
region -- 'RP-1' is the confirmed, correct target."
```

Step 3B -- No matching named set/surface exists: do NOT dispatch Coding Agent to guess

This is a hard rule, not a judgment call: if no named set or surface in the model corresponds to what the user described, Coding Agent must NOT be dispatched to select the region programmatically (e.g. via findAt(), bounding-box coordinate logic, or any other improvised geometric selection). Silently applying a load or BC to a programmatically-guessed region is a serious, hard-to-catch class of error -- the script may run successfully and produce a plausible-looking result while acting on entirely the wrong part of the model.

Instead: route_to_file_uploader, with a message_to_user that plainly and specifically explains what's needed -- name the kind of region required (a node set, element set, or surface, depending on the operation -- see WORKFLOW 2/3/4 below for which type each operation typically needs) and ask the user to define that set/surface in Abaqus CAE (or their pre-processor of choice) under a clear name, then re-upload the updated .inp file.

```
EXAMPLE message_to_user:
"To apply this load correctly, I need the specific surface it should act on to
be defined as a named surface in your model. Could you create a surface (in
Abaqus CAE: Assembly module > Tools > Surface > Create) covering the top face
of the bracket, give it a clear name, and re-upload the updated .inp file?"
```

Step 4 -- After re-upload, re-verify before proceeding

The automatic File Relevance Checker pass runs unconditionally after any upload, but it verifies file TYPE/relevance -- it does not check for a specific named set inside the file. On your next turn, re-run Step 2 yourself against the newly uploaded file's summary. If the needed set now exists, proceed per Step 3A. If it still doesn't exist or the name is unclear, you may request it once more, but treat this as a SOFT LIMIT: if this happens a second time on the same request, stop looping and use ask_user instead of another silent re-upload request -- state plainly and specifically what's still missing or ambiguous, since at that point the issue is more likely a genuine misunderstanding than a simple oversight.

> *Note: There is no automated loop-counter protecting this specific scenario the way FILE_RESOLUTION_LOOP_COUNT protects the earlier file-identification phase -- this re-verification happens during planning_and_dispatch, after files were already initially resolved. Use your own judgment to avoid looping indefinitely.*

WORKFLOW 1: Informational / Interpretive Questions

Task pattern: the user asks something about an existing model or existing results, with no request to change the model, run a new analysis, or produce a new file. Examples: "what material is my part made of", "what's the peak stress in my results", "explain the boundary conditions in step 2", "is this displacement reasonable for this load case".

Dispatch pattern

- Dispatch Subject Matter Expert (SME) with a clear, self-contained question and any relevant file context.
- SME answers using read_parsed_file_info and, where a visual would help confirm something, the two screenshot tools -- it does not write or run any code.
- If SME's answer contains a quantitative/numeric claim, Senior SME automatically and independently re-verifies it before it reaches you -- you do not need to dispatch this yourself, and you will only ever see the final, reviewed message.
- If SME reports it cannot answer because specific information isn't present in the parsed summaries at all (not an ambiguity issue, a genuine data gap) -- this is THE SME MISSING-INFORMATION LOOP: dispatch Coding Agent to extract EXACTLY the missing information (a script that reads the .odb/.inp directly and reports the specific value/data SME needs), then hand the result back to SME to complete its answer. Do not treat this as a failure -- it's a normal, expected pattern for a data point that summary parsing didn't happen to capture.

When Coding Agent is genuinely NOT needed

Most informational questions are fully answerable from read_parsed_file_info and screenshots alone -- resist the urge to write a script for something SME can already determine from the parsed summary. Reserve the missing-information loop for genuine gaps, not routine questions.

WORKFLOW 2: Applying Loads, Boundary Conditions, Connections, and Interactions

Task pattern: the user wants a load, boundary condition, tie/coupling constraint, or contact interaction applied to some part of the model. Examples: "apply a 500N downward force to the top of my bracket", "fix the base of the column", "tie these two parts together", "add a frictional contact between the pin and the hole".

Step 1 -- Apply THE GOLDEN RULE first

Before anything else, resolve the exact named region this operation targets, per THE GOLDEN RULE section above. Do not proceed to dispatch Coding Agent until the exact set/surface name is confirmed to exist.

Step 2 -- Confirm every other required value is established

Beyond the region, check whether the task fully specifies: the load/BC TYPE and MAGNITUDE/VALUE, the DIRECTION (if directional), and the analysis STEP it applies in. If any of these is missing and has no sensible default (e.g. the user said "apply a load" but never gave a magnitude or direction) -- this is a code-level/parameter input, per THE CODE-LEVEL INPUT RULE: collect it via Interface Agent (a number_input for magnitude, a selectbox for direction/step if there are a small number of valid choices), never via ask_user and never invented by Coding Agent.

Step 3 -- Decide whether to consult Consultant first

For a routine, well-understood operation (a simple pressure or concentrated force, a standard encastre or pinned/roller BC, a straightforward tie constraint between two clearly-matching surfaces), you may be confident enough to dispatch Coding Agent directly. For anything involving genuine engineering judgment -- contact interactions (friction coefficients, normal/tangential behavior choices), coupling constraints (kinematic vs. distributed), or any case where the physical setup could be represented multiple defensible ways -- consult Consultant first for the methodology, per the DYNAMIC DISPATCH default-toward-Consultant guidance.

Step 4 -- Dispatch Coding Agent

dispatch_instructions must be fully self-contained (Coding Agent has zero memory of anything outside this one dispatch): the exact verbatim region name(s), the operation type, magnitude/value and direction, which analysis step it applies in, WORKING_DIRECTORY restated, and any Consultant advice already collected, translated into plain instructions (never referencing Consultant by name).

Step 5 -- Verify the result once Coding Agent reports success

Coding Agent's final_success_message already names every file produced. For load/BC/connection tasks, it's often worth a quick visual confirmation -- dispatch SME (or use your own screenshot tool directly) to capture a screenshot with loads/BCs visible, confirming the operation was applied where intended, especially before finalizing a task where getting this wrong would be a meaningful engineering error.

WORKFLOW 3: Material and Section Assignment

Task pattern: the user wants a material and/or section assigned to a part or a specific region of a part. Examples: "assign steel to my bracket", "create a shell section 2mm thick and assign it to the plate", "the hub should be aluminum, the shaft should be steel".

Whole-part vs. region-specific

If the material/section applies to an ENTIRE part with a single section assignment, THE GOLDEN RULE does not apply -- Coding Agent can reference the whole part directly, no named set needed. If the assignment applies to only PART of a part (e.g. "the hub should be aluminum, the shaft should be steel" where both are modeled as one part), THE GOLDEN RULE DOES apply -- a named region/set is required for each distinct assignment, following the same decision procedure as WORKFLOW 2.

Missing material properties

If the user names a material generically ("steel", "aluminum") without specific property values (Young's modulus, Poisson's ratio, density), and the task requires exact values rather than a reasonable engineering default: this is a code-level input -- collect via Interface Agent, or if Consultant has already suggested reasonable standard values for the named material as part of its advice, relay those values explicitly in dispatch_instructions rather than asking the user for numbers Consultant already provided.

WORKFLOW 4: Meshing

Task pattern: the user wants the model (or part of it) meshed. Examples: "mesh my part with 5mm elements", "refine the mesh near the fillet", "use a finer mesh on the hole surface".

Whole-part meshing

If meshing applies uniformly to an entire part with one global seed size, THE GOLDEN RULE does not apply -- Coding Agent can seed and mesh the whole part directly.

Region-specific mesh refinement

If the task asks for a different mesh density/control on a SPECIFIC sub-region ("finer near the fillet", "coarser on the outer edge"), THE GOLDEN RULE applies -- the target region needs to be a named set/edge/face reference, following the same decision procedure as WORKFLOW 2.

Missing mesh size

If no element size is given and the task requires a specific value rather than a reasonable default -- collect via Interface Agent (a number_input with sensible min/max/step for the model's scale).

Verifying the result

After Coding Agent reports success, a mesh screenshot (capture_abaqus_model_screenshot with show_mesh enabled) is a fast, cheap way to visually confirm the mesh was applied as expected before finalizing.

WORKFLOW 5: Job Submission / Running the Analysis

Task pattern: the model is already fully set up (materials, sections, steps, loads, BCs, mesh all in place) and the user wants the analysis actually run. Examples: "run the analysis", "submit the job and give me the results".

Dispatch pattern

This is typically a direct Coding Agent dispatch -- writing and submitting the job is a well-understood, routine scripting operation. Consultant consultation is rarely needed here unless the job needs specific non-default solver settings (increment control, nonlinear geometry settings) that the user hasn't specified and that require engineering judgment to choose sensibly.

If the model setup is incomplete

Before dispatching a job-submission task, confirm (via read_parsed_file_info on the .inp summary, or by dispatching SME) that the model actually has the necessary steps, loads, BCs, and mesh already defined. If something is missing, this is really a WORKFLOW 2/3/4-type task first -- resolve that first, then submit.

Handling execution failures

Job-submission scripts fail for the same reasons any Coding Agent script can fail (see the ESCALATION HANDLING and GIVE-UP THRESHOLD sections of your own system prompt) -- no special handling beyond the existing escalation mechanics is needed here. If Execution Agent escalates after repeated failures on a job submission specifically, a genuinely different approach (e.g. checking with Consultant on possible solver/ convergence issues) is often more productive than another direct resubmission attempt.

WORKFLOW 6: Post-Processing and Custom Results Extraction

Task pattern: the user wants a specific result, plot, or derived quantity from completed results. Examples: "give me a stress-vs-time plot for this node", "extract the reaction forces at the support", "what's the maximum displacement over the whole run".

Step 1 -- Check if SME can already answer from the odb_summary

Many results questions are answerable directly from odb_summary.md via SME -- try this first, per WORKFLOW 1, before assuming a new script is needed.

Step 2 -- If the summary doesn't have it, this is the missing-information loop

If SME reports the specific data genuinely isn't captured in the parsed summary (a custom derived quantity, a specific node/element not individually broken out), dispatch Coding Agent to write an extraction script -- same pattern as WORKFLOW 1's missing-information loop.

Step 3 -- If the extraction targets a specific named set, THE GOLDEN RULE applies

If the requested result is for a SPECIFIC node/element set ("the reaction force at RP-1", "stress in the fillet region"), confirm that set exists (in the .inp, and therefore in the .odb, since .odb sets mirror what was defined when the job ran) before dispatching the extraction script, exactly as in WORKFLOW 2.

Edge case: the requested region was never part of the original run

If the user wants a result for a region that was NOT defined as a set at the time the analysis was run, this cannot be recovered by a smarter extraction script alone -- the .odb simply doesn't contain that data. This requires: (1) defining the set in the model, (2) re-running the analysis, THEN (3) extracting the result. Recognize this case explicitly and mark the plan step needs_replanning rather than dispatching Coding Agent to attempt an extraction that cannot succeed -- explain to the user plainly that this requires re-running the analysis with the region properly defined first.

WORKFLOW 7: Multi-File Comparison Tasks

Task pattern: the user wants two or more files compared -- multiple .odb runs, or a model vs. its results. Examples: "compare the stresses between these two runs", "how does my updated model differ from the original".

File resolution

Multi-file requirements are resolved during IDENTIFYING_FILES per your own phase logic (named files, a stated count, or an open-ended "all my runs" set) -- by the time you reach planning_and_dispatch, every required file should already be confirmed present and relevant.

Dispatch pattern

- For a comparison that's mostly interpretive ("which run has lower peak stress") -- dispatch SME, giving it all relevant file paths in one self-contained dispatch; it can read multiple summaries in one turn.
- For a comparison that needs a generated artifact (a comparison plot, a side-by-side table) -- dispatch Coding Agent to produce it, with all relevant file paths given explicitly.
- If the comparison result should be shown to the user as a structured widget (a table or chart) rather than only described in text, dispatch_interface_agent with an OUTPUT widget (table, dataframe, line_chart, or bar_chart as appropriate) once the data is ready.

WORKFLOW 8: Geometry-to-Model Setup from CAD Files (.step/.stp)

Task pattern: the user uploads raw CAD geometry (.step/.stp) and wants a working Abaqus model built from it -- import geometry, define materials/sections, mesh, set up steps, apply loads/BCs, and often run the analysis. This is the most involved task category and typically spans MULTIPLE plan steps.

Why this is different

A .step/.stp file contains ONLY geometry -- no materials, mesh, loads, or boundary conditions (see FILE-TYPE KNOWLEDGE in your own system prompt). Every other workflow in this document assumes at least a partially-set-up .inp model already exists; this workflow starts from nothing but geometry.

Recommended sequencing

- Step 1: Import the geometry and create the Abaqus part -- straightforward Coding Agent dispatch.
- Step 2: Consult Consultant early for the overall approach -- given the breadth of decisions involved (mesh strategy, whether the geometry needs simplification/defeaturing, section/material approach), this is exactly the case where Consultant's default involvement is most valuable, not something to skip.
- Step 3: Material and section assignment -- WORKFLOW 3 applies (whole-part or region-specific, per that workflow's own distinction).
- Step 4: Meshing -- WORKFLOW 4 applies.
- Step 5: Steps, loads, and boundary conditions -- WORKFLOW 2 applies, including THE GOLDEN RULE. Since the geometry has just been imported, named sets/surfaces will generally NOT exist yet -- expect to need the file-re-upload cycle from THE GOLDEN RULE at this stage, or have Coding Agent define the needed named sets/surfaces itself AS PART of the same script that also applies the operation, if the exact target geometry is unambiguous from the user's description and confirmed visually (e.g. via a screenshot) rather than requiring a fresh upload cycle. Use judgment: THE GOLDEN RULE's concern is avoiding an ambiguous or guessed selection, not necessarily forcing a file round-trip when the target is genuinely unambiguous and Coding Agent is DEFINING the set explicitly and verifiably as its own step, rather than silently guessing.
- Step 6: Job submission -- WORKFLOW 5 applies.

Note: The plan for this category of task will typically have 4-6 steps, each potentially needing a different subagent. Build this out explicitly in `plan` from the start once procedural_rag_tool confirms this is the matching category, rather than discovering the full scope one step at a time.

CROSS-CUTTING GUIDANCE

When to consult Consultant vs. dispatch Coding Agent directly

Your own system prompt already establishes the default: consult Consultant unless you're genuinely confident in the approach. Across the workflows above, the clearest signals for genuine confidence are: the operation is routine and well-understood (a standard load/BC type, a whole-part material assignment, a simple mesh), OR Consultant has already advised on this exact kind of operation earlier in the same task, OR this document's guidance for the matching workflow already gives you enough procedural clarity. The clearest signals FOR consulting Consultant: multiple valid approaches exist (contact/interaction setup, coupling type choices), the task is a WORKFLOW 8-style multi-step build, or the task involves quantities/assumptions with real engineering judgment behind them (mesh convergence, nonlinearity, solver settings).

Interface Agent parameter collection, across all workflows

Every workflow above that mentions "collect via Interface Agent" follows the same protocol already defined in THE INTERFACE AGENT PROTOCOL and THE CODE-LEVEL INPUT RULE in your own system prompt -- pick the right widget type for the value being collected (number_input for a magnitude, selectbox for a bounded set of named choices, checkbox for a yes/no), always include a clear "help" description with an example value, and relay the collected value explicitly into Coding Agent's next dispatch_instructions -- it has no other way to see it.

Handling Coding Agent questions and Execution escalations, across all workflows

If, in the course of ANY workflow above, Coding Agent reports back with an empty final_success_message (a genuine question or code-level input need) -- resolve it per HANDLING CODING AGENT'S QUESTIONS AND INPUT REQUESTS in your own system prompt before re-dispatching. If Execution Agent escalates after repeated consecutive failures -- read the difficulty summary and suggestion, and seriously consider whether the actual problem is one THIS document could have caught earlier (most commonly: a named region that doesn't actually exist, or a mismatch between the exact name given to Coding Agent and what's actually in the file) before assuming a fundamentally different approach is needed.

What this document does NOT cover

General chat, casual remarks, and questions about the copilot itself are handled entirely by your own IDENTIFYING_TASK phase logic -- this document has no bearing on that phase. This document applies once a real task has been identified and you are in (or about to enter) planning_and_dispatch.

A reminder that applies to every workflow above

Never reveal internal agent names, node names, tool names, or system architecture to the user in message_to_user, under any framing -- this constraint from your own system prompt is not suspended for any scenario described in this document.